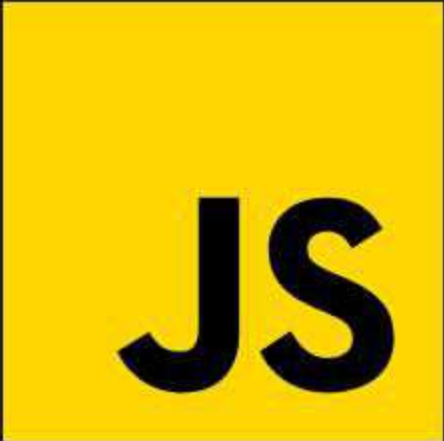




JS

6 Promise Use Cases In JavaScript



@richwebdeveloper



1. Loading Data from an API



```
1 fetch('https://api.example.com/data')  
2   .then(response => response.json())  
3   .then(data => console.log(data))  
4   .catch(error => console.error("Failed to load data", error));
```

Use Case: Fetch data from a remote server and process it when available.



@richwebdeveloper



2. Waiting for Multiple Requests to Complete



```
1  Promise.all([
2    fetch('https://api.example.com/data1'),
3    fetch('https://api.example.com/data2')
4  ]).then(responses => {
5    return Promise.all(responses.map(res => res.json()));
6  }).then(data => console.log(data));
```

Use Case: Execute multiple API calls simultaneously and wait for all of them to complete.



@richwebdeveloper



3. User Authentication

```
1 function loginUser(email, password) {  
2   return new Promise((resolve, reject) => {  
3     setTimeout(() => {  
4       if (email === 'user@example.com' && password === 'password123') {  
5         resolve({ userId: 1, profile: "User Profile" });  
6       } else {  
7         reject(new Error("Authentication failed"));  
8       }  
9     }, 1000);  
10   });  
11 }
```

Use Case: Simulate user login, resolving on success and rejecting on failure.



@richwebdeveloper



4. Data Processing Pipeline

```
1 fetch('https://api.example.com/data')
2   .then(response => response.json())
3   .then(data => processData(data))
4   .then(processedData => console.log(processedData))
5   .catch(error => console.error("Failed in processing pipeline", error));
6
7 function processData(data) {
8   // Process data
9   return modifiedData;
10 }
```

Use Case: Chain operations where each step depends on the successful completion of the previous one.



@richwebdeveloper



5. Image Loading



```
1 function loadImage(url) {  
2   return new Promise((resolve, reject) => {  
3     const image = new Image();  
4     image.onload = () => resolve(image);  
5     image.onerror = () => reject(new Error('Failed to load image'));  
6     image.src = url;  
7   });  
8 }
```

Use Case: Load an image and perform actions once it is fully loaded or handle errors if it fails to load.



@new_javascript



6. Timeouts



```
1 function delay(ms) {  
2   return new Promise(resolve => setTimeout(resolve, ms));  
3 }  
4  
5 delay(1000).then(() => console.log("Delayed for 1 second"));
```

Use Case: Execute some action after a delay, without blocking the main thread.



@richwebdeveloper

